COTRTUPC-cbl

Report date: *2026-10-29T17:39:55.639725330Z*

User ID: *374199896615:transform-demo*

AWS Account ID: *374199896615*

Table of Contents

1 High level overview....2
1.1 Overview....2
1.2 Key features....2
2 Input output....3
3 Data flow and dependencies....3
3.1 Data flow....3
3.2 Data dependencies....6
3.3 External system integrations....7
4 Error handling....7
5 Screen interface....9
5.1 Overview....9
5.2 User input interface....9
5.3 Pf keys....10

AWS Transform uses generative AI. Use of AWS Transform is subject to AWS Responsible AI Policy https://aws.amazon.com/ai/responsible-ai/policy/

Documentation

1. High level overview

1.1. Overview

This program is part of a credit card management system (CardDemo), operating within the administrative business layer responsible for maintaining reference data used to classify financial transactions. It serves as a maintenance screen handler for transaction type codes, which are foundational lookup values used across the card processing workflow to categorize and describe transaction activity. The program accepts user input via a CICS terminal screen, processes requests to view, create, update, or delete transaction type records in the CARDDEMO.TRANSACTION_TYPE database table, and guides the user through a structured multi-step confirmation workflow to ensure data integrity before committing changes.

1.2. Key features

Feature	Description
Transaction Type Lookup	Retrieves an existing transaction type record from the database by its 2-digit numeric type code and displays it to the user for review or modification.
Record Update	Allows authorized users to modify the description of an existing transaction type, with a two-step validation and confirmation process before committing changes to the database.
New Record Creation	When a searched transaction type code is not found, prompts the user to create a new record with the entered code and description via an INSERT operation.
Record Deletion	Supports deletion of a transaction type record with a mandatory confirmation step (F4 key), including handling of referential integrity errors when child records exist.
Input Validation	Validates that the transaction type code is a non-zero numeric value and that the description is a non-blank alphanumeric field before allowing any database operation.
Change Detection	Compares newly entered values against the originally fetched record to detect whether any actual changes were made, preventing unnecessary database updates.
PF Key Navigation Control	Manages which function keys (F3, F4, F5, F12, Enter) are active and valid based on the current processing state, providing context-sensitive screen navigation.
Multi-Step Workflow State Management	Maintains processing state across multiple screen interactions using a COMMAREA, tracking stages such as details-not-fetched, show-details, confirm-delete, changes-ok, and update-done.
Error Handling and Abend Recovery	Captures SQL errors with descriptive messages, handles lock conflicts and failed updates gracefully, and provides a structured abend routine for unexpected failures.

Feature	Description
Program Navigation	Supports returning to the calling admin program (COADM01C) or transaction list program (COTRTLIC) via F3, maintaining the calling context through the shared COMMAREA.

2. Input output

Name	Type	Description
CARDDEMO. TRANSACTION_TYPE	Read/Write	DB2 table storing transaction type codes and descriptions. Read to fetch existing transaction type details for display and validation. Updated to modify TR_DESCRIPTION for a given TR_TYPE. Inserted when a new transaction type record is created. Deleted when a transaction type record is removed.

3. Data flow and dependencies

3.1. Data flow

Step	Process	Description	Records
1	Program Initialization	Initializes working storage areas, sets transaction ID context, clears error messages, and establishes CICS abend handler	[WS-TRANID, CC-WORK-AREA, WS-MISC-STORAGE, WS-COMMAREA]
2	Commarea Restoration	Checks EIBCALEN to determine if this is a fresh entry or re-entry. If re-entry, restores CARDDEMO-COMMAREA and WS-THIS-PROGCOMMAREA from DFHCOMMAREA. If fresh entry or coming from admin/list program without re-enter flag, initializes both areas.	[DFHCOMMAREA, CARDDEMO-COMMAREA, WS-THIS-PROGCOMMAREA, TTUP-CHANGE-ACTION, CDEMO-FROM-PROGRAM, CDEMO-PGM-REENTER]
3	PF Key Storage and Validation	Stores the current PF key pressed via CSSTRPFY copybook routine, then validates whether the pressed key is valid in the current program context. Sets PFK-VALID or PFK-INVALID flag accordingly.	[WS-PFK-FLAG, CCARD-AID-PFK03, CCARD-AID-PFK04, CCARD-AID-PFK05, CCARD-AID-PFK12, CCARD-AID-ENTER, TTUP-CHANGE-ACTION]
4	Context State Reset Evaluation	Evaluates specific combinations of PF key and current state flags to determine if the program should reset to initial entry state (e.g., after completed update, failed changes, completed delete, or F12 cancel from certain states).	[CCARD-AID-PFK12, TTUP-CHANGE-ACTION, TTUP-OLD-DETAILS, CDEMO-PGM-ENTER, TTUP-DETAILS-NOT-FETCHED]
5	Main Action Dispatch	Evaluates the combination of PF key pressed and current program state to route to the appropriate processing branch: exit (F3), initial display, delete confirmation (F4), new record creation (F5), save	[CCARD-AID-PFK03, CCARD-AID-PFK04, CCARD-AID-PFK05, CCARD-AID-PFK12, TTUP-CHANGE-ACTION, CDEMO-TO-PROGRAM, CDEMO-TO-TRANID]

Step	Process	Description	Records
		confirmed changes (F5), cancel (F12), invalid key handling, or general input processing.	
6	Exit to Calling Program (F3)	When F3 is pressed, determines the target program and transaction ID from CDEMO-FROM-PROGRAM/TRANID or defaults to admin program. Issues CICS SYNCPOINT then XCTL to transfer control.	[CDEMO-FROM-TRANID, CDEMO-FROM-PROGRAM, CDEMO-TO-TRANID, CDEMO-TO-PROGRAM, LIT-ADMINPGM, LIT-ADMINTRANID, CARDDEMO-COMMAREA]
7	Receive Map Input	Issues CICS RECEIVE MAP to read user input from the CTRTUPA map in COTRTUP mapset into CTRTUPAI input area.	[CTRTUPAI, TRTYPCDI, TRTYDSCI, WS-RESP-CD, WS-REAS-CD]
8	Store Map Input to New Details	Transfers received map field values into TTUP-NEW-DETAILS working storage. Handles asterisk and space as low-value indicators. Trims transaction type code and description before storing.	[TRTYPCDI, TRTYDSCI, TTUP-NEW-TTYP-TYPE, TTUP-NEW-TTYP-TYPE-DESC, TTUP-NEW-DETAILS]
9	Edit and Validate Map Inputs	Validates the transaction type code (numeric, required, non-zero, 2 digits) and description (alphanumeric, required, up to 50 chars). Compares new values against old values to detect changes. Sets INPUT-OK or INPUT-ERROR flags and appropriate state flags.	[TTUP-NEW-TTYP-TYPE, TTUP-NEW-TTYP-TYPE-DESC, TTUP-OLD-TTYP-TYPE, TTUP-OLD-TTYP-TYPE-DESC, WS-EDIT-TTYP-FLAG, WS-EDIT-DESC-FLAGS, WS-INPUT-FLAG, WS-RETURN-MSG]
10	Decide Action Based on State	Evaluates current program state flags to determine next action: fetch record from DB if not fetched, handle cancel/F12 by re-reading DB, confirm delete state, validate changes for update confirmation, handle backed-out changes, handle new record creation, or handle completed updates.	[TTUP-CHANGE-ACTION, CCARD-AID-PFK12, FLG-TRANFILTER-ISVALID, FOUND-TRANTYPE-IN-TABLE, INPUT-ERROR, NO-CHANGES-DETECTED]
11	Read Transaction Type from Database	Executes SQL SELECT against CARDDEMO. TRANSACTION_TYPE table using TR_TYPE as key. Stores result in DCL-TR-TYPE and DCL-TR-DESCRIPTION. Sets FOUND-TRANTYPE-IN-TABLE flag on success, FLG-TRANFILTER-NOT-OK on not-found or SQL error.	[DCL-TR-TYPE, DCL-TR-DESCRIPTION, TTUP-NEW-TTYP-TYPE, SQLCODE, WS-DISP-SQLCODE, SQLERRM]
12	Store Fetched Data to Old Details	Copies the SQL-fetched transaction type and description into TTUP-OLD-DETAILS for later comparison and display purposes.	[DCL-TR-TYPE, DCL-TR-DESCRIPTION-TEXT, DCL-TR-DESCRIPTION-LEN, TTUP-OLD-TTYP-TYPE, TTUP-OLD-TTYP-TYPE-DESC]
13	Write Processing - Update or Insert	Attempts SQL UPDATE on CARDDEMO. TRANSACTION_TYPE. If SQLCODE=+100 (not found), falls through to INSERT. On SQLCODE=0, issues CICS SYNCPOINT. On lock error (-911) or other negative SQLCODE, sets appropriate failure flags.	[DCL-TR-TYPE, DCL-TR-DESCRIPTION, DCL-TR-DESCRIPTION-TEXT, DCL-TR-DESCRIPTION-LEN, TTUP-NEW-TTYP-TYPE, TTUP-NEW-TTYP-TYPE-DESC, SQLCODE, WS-DISP-SQLCODE, TTUP-CHANGE-ACTION]

Step	Process	Description	Records
14	Insert New Record	When UPDATE finds no matching row (SQLCODE=+100), executes SQL INSERT INTO CARDDEMO.TRANSACTION_TYPE with TR_TYPE and TR_DESCRIPTION. Issues CICS SYNCPOINT on success, sets TABLE-UPDATE-FAILED on error.	[DCL-TR-TYPE, DCL-TR-DESCRIPTION, SQLCODE, WS-DISP-SQLCODE, SQLERRM, WS-RETURN-MSG]
15	Delete Processing	Executes SQL DELETE FROM CARDDEMO. TRANSACTION_TYPE WHERE TR_TYPE matches old type. On success sets TTUP-DELETE-DONE and issues SYNCPOINT. On foreign key violation (-532) or other error, sets RECORD-DELETE-FAILED and TTUP-DELETE-FAILED with error message.	[TTUP-OLD-TTYP-TYPE, DCL-TR-TYPE, SQLCODE, WS-DISP-SQLCODE, SQLERRM, WS-RETURN-MSG, TTUP-CHANGE-ACTION]
16	Screen Initialization and Setup	Initializes output map CTRTUPAO with low values, populates screen titles from COTTL01Y copybook, sets current date and time, program name, and transaction ID in screen header fields.	[CTRTUPAO, CCDA-TITLE01, CCDA-TITLE02, WS-CURDATE-DATA, WS-CURDATE-MM-DD-YY, WS-CURTIME-HH-MM-SS, TRNNAMEO, PGMNAMEO, CURDATEO, CURTIMEO]
17	Setup Screen Variables	Based on current state, populates screen output fields with either initial blank values, original fetched values (TTUP-OLD-DETAILS), or updated new values (TTUP-NEW-DETAILS) for transaction type code and description.	[TRTYPCDO, TRTYDSCO, TTUP-OLD-TTYP-TYPE, TTUP-OLD-TTYP-TYPE-DESC, TTUP-NEW-TTYP-TYPE, TTUP-NEW-TTYP-TYPE-DESC, CTRTUPAO]
18	Setup Information and Error Messages	Sets the appropriate informational message (WS-INFO-MSG) based on current state, center-justifies it, and moves it to INFOMSGO. Also moves WS-RETURN-MSG to ERRMSGO for error display.	[WS-INFO-MSG, WS-STRING-LEN, WS-STRING-MID, WS-STRING-OUT, INFOMSGO, ERRMSGO, CTRTUPAO]
19	Setup Screen Field Attributes	Sets field protection attributes: first protects all fields, then selectively unprotects based on state. Sets cursor position, field colors (red for errors), and uses CSSETATY copybook for description field attribute management.	[TRTYPCDA, TRTYDSCA, INFOMSGA, TRTYPCDL, TRTYDSCL, TRTYPCDC, DFHBMPRF, DFHBMFSE, DFHRED, CTRTUPAI]
20	Setup PF Key Attributes	Sets display attributes for PF key indicators on screen based on which keys are valid in the current state. Uses DFHBMASB for active keys and DFHBMDAR for inactive keys.	[FKEYSA, FKEY04A, FKEY05A, FKEY12A, CTRTUPAI, DFHBMASB, DFHBMDAR]
21	Send Screen to Terminal	Issues CICS SEND MAP to display the populated CTRTUPAO output map to the user terminal with CURSOR, ERASE, and FREEKB options.	[CTRTUPAO, CCARD-NEXT-MAP, CCARD-NEXT-MAPSET, WS-RESP-CD]
22	Common Return	Packages CARDDEMO-COMMAREA and WS-THIS-PROGCOMMAREA into WS-COMMAREA, moves return message to CCARD-ERROR-MSG, then issues CICS RETURN with transaction ID and commarea to await next user interaction.	[WS-RETURN-MSG, CCARD-ERROR-MSG, CARDDEMO-COMMAREA, WS-THIS-PROGCOMMAREA, WS-COMMAREA, LIT-THISTRANID]

3.2. Data dependencies

Dependency	Description
EIBCALEN commarea length check	Program behavior on entry depends on EIBCALEN being non-zero to restore prior state. Zero length forces full re-initialization, losing any prior context.
TTUP-CHANGE-ACTION state machine	The entire program flow is governed by the TTUP-CHANGE-ACTION flag acting as a state machine. Each state (not-fetched, show-details, confirm-delete, changes-ok, etc.) determines valid PF keys, screen field protection, and available database operations. Incorrect state transitions cause abend via ABEND-ROUTINE.
CDEMO-PGM-REENTER vs CDEMO-PGM-ENTER flag	Distinguishes between first-time entry and re-entry after user interaction. Controls whether commarea is restored or initialized, and whether screen setup skips certain initialization steps.
CDEMO-FROM-PROGRAM calling context	The originating program (LIT-ADMINPGM=COADM01C or LIT-LISTTPGM=COTRTLIC) combined with CDEMO-PGM-REENTER determines whether to initialize state or restore it. F3 exit destination also depends on this value.
SQL SELECT before UPDATE/ DELETE	The 9000-READ-TRANTYPE must successfully execute before any update or delete operation. TTUP-OLD-DETAILS populated from the SELECT is used as the WHERE key for DELETE and as the baseline for change comparison. No optimistic locking token is used.
TTUP-OLD-DETAILS vs TTUP-NEW-DETAILS change detection	The 1205-COMPARE-OLD-NEW paragraph compares old and new values using UPPER-CASE and TRIM functions. If no change is detected, the update path is bypassed and NO-CHANGES-DETECTED message is shown. Both type and description must differ to trigger update flow.
FLG-TRANFILTER-ISVALID prerequisite for DB operations	Database read (9000-READ-TRANTYPE) and subsequent store operations only proceed if FLG-TRANFILTER-ISVALID is set. The flag is set only after successful numeric validation of the transaction type code in 1210-EDIT-TRANTYPE.
TTUP-CHANGES-OK-NOT-CONFIRMED two-step confirmation	Updates require a two-step user confirmation: first F5 validates inputs and sets TTUP-CHANGES-OK-NOT-CONFIRMED, then a second F5 press triggers the actual SQL UPDATE/ INSERT. This prevents accidental data modification.
TTUP-CONFIRM-DELETE two-step delete confirmation	Delete requires F4 to set TTUP-CONFIRM-DELETE state (prompting user), then a second F4 press triggers actual SQL DELETE. F12 at confirm-delete state cancels the operation.
SQL SQLCODE=-532 foreign key constraint on delete	Delete operation explicitly handles SQLCODE -532 (foreign key violation), indicating child records exist in related tables. Program sets RECORD-DELETE-FAILED and instructs user to delete child records first, preventing orphaned data.
SQLCODE=-911 lock contention on update	Update operation handles SQLCODE -911 (deadlock/timeout) separately from other errors, setting TTUP-CHANGES-OKAYED-LOCK-ERROR to inform user the record could not be locked, allowing retry.
CICS SYNCPOINT after successful DML	EXEC CICS SYNCPOINT is issued immediately after successful INSERT, UPDATE, or DELETE to commit the unit of work. Failure paths do not issue SYNCPOINT, implying implicit rollback on CICS RETURN.

Dependency	Description
WS-RETURN-MSG-OFF guard for error messages	Multiple validation and error-setting routines check WS-RETURN-MSG-OFF before overwriting the error message, ensuring only the first error encountered is displayed to the user.
DCLTRTYP SQL host variable structure	The DCL-TR-TYPE and DCL-TR-DESCRIPTION host variables declared via DCLTRTYP copybook must match the CARDDEMO.TRANSACTION_TYPE table column definitions exactly. DCL-TR-DESCRIPTION uses a VARCHAR structure with separate length (DCL-TR-DESCRIPTION-LEN) and text (DCL-TR-DESCRIPTION-TEXT) components.
DCLTRCAT included but not directly used in visible code	EXEC SQL INCLUDE DCLTRCAT is present, declaring transaction category table host variables. While not directly referenced in the visible procedure division, its inclusion suggests potential future use or shared host variable namespace dependency.
Commarea size constraint	WS-COMMAREA is defined as PIC X(2000). The combined length of CARDDEMO-COMMAREA and WS-THIS-PROGCOMMAREA must not exceed 2000 bytes. Overflow would cause data truncation in the CICS RETURN commarea.
CSSETATY copybook attribute setting for description field	Screen attribute setup for the TRTYDSC field is delegated to the CSSETATY copybook via COPY REPLACING. This copybook's logic for setting normal/error color attributes depends on FLG-DESCRIPTION-ISVALID, FLG-DESCRIPTION-NOT-OK, and FLG-DESCRIPTION-BLANK flags being correctly set before 3300-SETUP-SCREEN-ATTRS is called.

3.3. External system integrations

System	Type	Purpose
IBM CICS	Online Transaction Processing (OLTP) middleware	Provides the runtime environment for the program. Used for: terminal map send/receive (SEND MAP, RECEIVE MAP), program-to-program transfer (XCTL), transaction return with commarea (RETURN), unit-of-work commit (SYNCPOINT), abend handling (HANDLE ABEND, ABEND), and terminal output for abend messages (SEND). Transaction ID CTTU is registered with CICS.
IBM DB2	Relational Database Management System	Persistent storage for transaction type data. Program performs SELECT, UPDATE, INSERT, and DELETE operations against the CARDDEMO.TRANSACTION_TYPE table. SQLCA is used for return code and error message retrieval. Host variables are declared via DCLTRTYP and DCLTRCAT copybooks.

4. Error handling

Category	Description
Input Validation Errors	Validates transaction type code (numeric, non-zero, required) and description (alphanumeric, required) via dedicated edit routines (1210-EDIT-TRANTYPE, 1230-EDIT-ALPHANUM-REQD, 1245-EDIT-NUM-REQD). Sets INPUT-ERROR flag and populates WS-RETURN-MSG with field-specific messages only if no prior message exists (WS-RETURN-MSG-OFF guard). Invalid search keys set TTUP-INVALID-SEARCH-KEYS state; blank keys set TTUP-DETAILS-NOT-FETCHED. Cursor is repositioned to the offending field. Screen attributes (red color via DFHRED) highlight erroneous fields.

Category	Description
PF Key / Navigation Errors	Invalid PF key presses are detected in 0001-CHECK-PFKEYS by evaluating CCARD-AID against context-valid keys. Sets PFK-INVALID flag and WS-INVALID-KEY-PRESSED message. The EVALUATE in 0000-MAIN routes WS-INVALID-KEY-PRESSED directly to re-send the map without further processing, preserving current screen state.
Database Read Errors (SQL SELECT)	In 9100-GET-TRANSACTION-TYPE, SQLCODE is evaluated after SELECT. SQLCODE=0 sets FOUND-TRANTYPE-IN-TABLE. SQLCODE=+100 (not found) sets INPUT-ERROR, FLG-TRANFILTER-NOT-OK, and WS-RECORD-NOT-FOUND message. Negative SQLCODE sets INPUT-ERROR, FLG-TRANFILTER-NOT-OK, and builds a detailed error string including table name, SQLCODE value, and SQLERRM into WS-RETURN-MSG.
Database Update Errors (SQL UPDATE)	In 9600-WRITE-PROCESSING, SQLCODE is evaluated after UPDATE. SQLCODE=0 issues CICS SYNCPOINT. SQLCODE=+100 (row not found) triggers 9700-INSERT-RECORD as a fallback insert. SQLCODE=-911 (lock conflict) sets COULD-NOT-LOCK-REC-FOR-UPDATE message and TTUP-CHANGES-OKAYED-LOCK-ERROR state. Other negative SQLCODEs set TABLE-UPDATE-FAILED with a detailed error string. State flags (TTUP-CHANGES-OKAYED-LOCK-ERROR, TTUP-CHANGES-OKAYED-BUT-FAILED) drive subsequent screen messaging.
Database Insert Errors (SQL INSERT)	In 9700-INSERT-RECORD, SQLCODE=0 issues CICS SYNCPOINT. Any other SQLCODE sets TABLE-UPDATE-FAILED and builds an error string with table name, SQLCODE, and SQLERRM into WS-RETURN-MSG, then exits the routine. The calling paragraph maps TABLE-UPDATE-FAILED to TTUP-CHANGES-OKAYED-BUT-FAILED state.
Database Delete Errors (SQL DELETE)	In 9800-DELETE-PROCESSING, SQLCODE=0 sets TTUP-DELETE-DONE and issues SYNCPOINT. SQLCODE=-532 (referential integrity violation) sets RECORD-DELETE-FAILED with a message instructing deletion of child records first, including SQLCODE and SQLERRM. Other negative SQLCODEs set RECORD-DELETE-FAILED and TTUP-DELETE-FAILED with a generic failure message including SQLCODE and SQLERRM.
Unexpected / Unhandled State Errors	In 2000-DECIDE-ACTION, a WHEN OTHER clause handles unexpected program state by populating ABEND-CULPRIT, ABEND-CODE ('0001'), and ABEND-MSG ('UNEXPECTED DATA SCENARIO'), then performing ABEND-ROUTINE. ABEND-ROUTINE sends the abend data to the screen, cancels the CICS HANDLE ABEND, and issues CICS ABEND with the abend code, ensuring controlled termination.
CICS Abend Handling	At program entry in 0000-MAIN, EXEC CICS HANDLE ABEND LABEL(ABEND-ROUTINE) is registered to intercept any unexpected CICS abend. ABEND-ROUTINE sets a default message if none exists, assigns program name and code '9999', sends abend data to the terminal with ERASE, cancels the abend handler, and re-abends with CICS ABEND to ensure proper CICS transaction termination.
No-Change Detection	In 1205-COMPARE-OLD-NEW, old and new values are compared case-insensitively including trimmed length. If identical, NO-CHANGES-DETECTED message is set in WS-RETURN-MSG (guarded by WS-RETURN-MSG-OFF) and NO-CHANGES-FOUND flag is set, preventing unnecessary database updates and prompting the user that no changes were detected.
Cancellation and Rollback Handling	User-initiated cancellations via PF12 are handled contextually: cancelling a pending update sets WS-UPDATE-WAS-CANCELLED and TTUP-CHANGES-BACKED-OUT; cancelling a delete sets WS-DELETE-WAS-CANCELLED and TTUP-DETAILS-NOT-FETCHED. No

Category	Description
	explicit SQL ROLLBACK is issued; CICS implicit rollback applies since SYNCPOINT is only called on success.

5. Screen interface

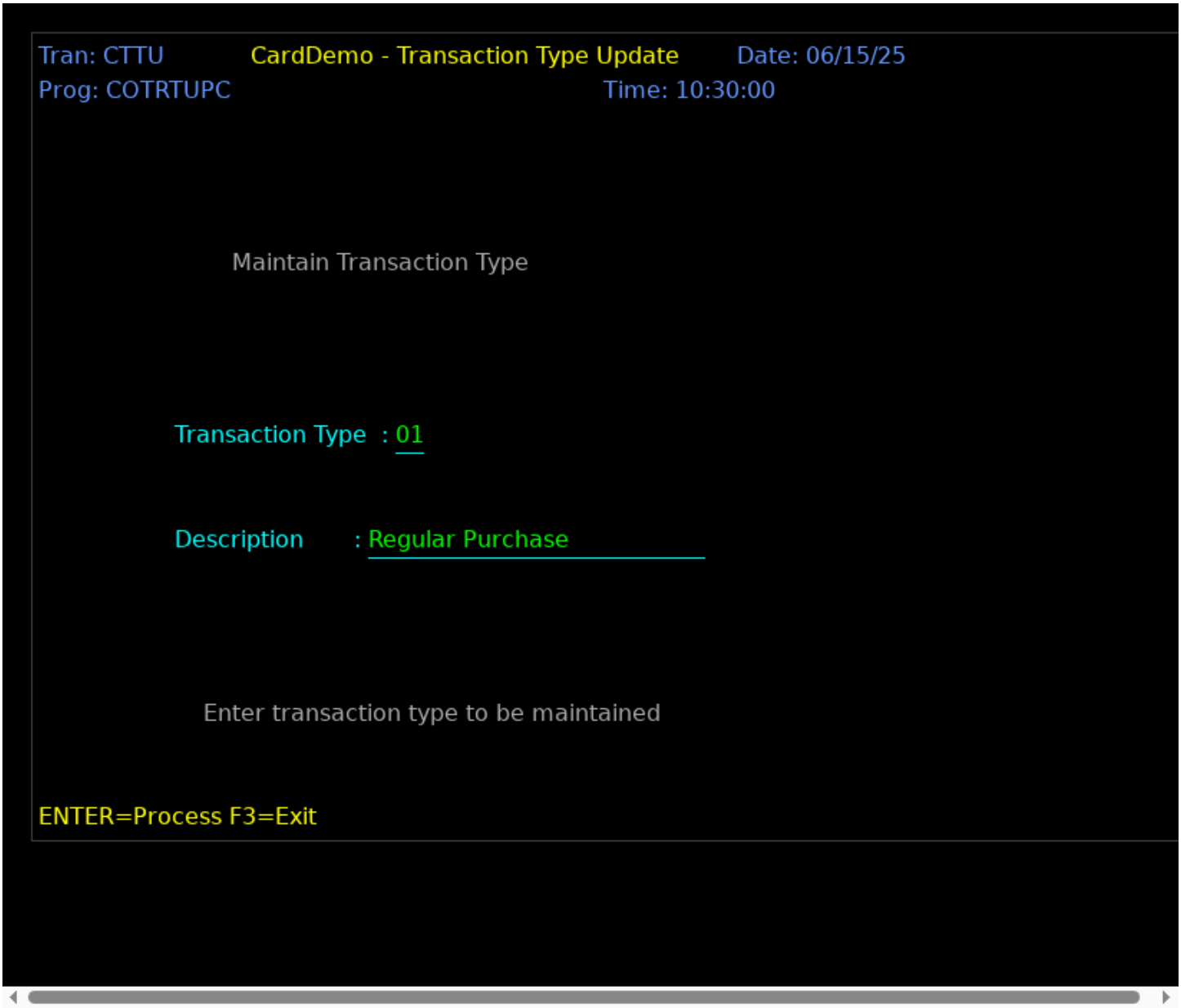
5.1. Overview

Mapset Name: COTRTUP

Map Name: CTRTUPA

Business Function: Provides a CICS 3270 terminal interface for maintaining (viewing, adding, updating, and deleting) Transaction Type records in the CardDemo application. The screen allows an administrator to look up a transaction type by its 2-digit numeric code, view its description, modify the description, confirm changes before saving, or delete the record. It interacts with the CARDDEMO.TRANSACTION_TYPE database table and supports full CRUD operations with validation and confirmation workflows.

5.2. User input interface



5.3. Pf keys

Key	Action
ENTER	Process the current screen input — reads the transaction type code entered, validates it, fetches the record from the database, and determines the next action based on current context (search, edit, etc.)
F3	Exit the Transaction Type Update screen and transfer control back to the calling program (Admin menu COADM01C or list screen COTRTLIC), issuing a SYNCPOINT before the XCTL
F4	Initiate delete of the currently displayed transaction type record. First press prompts for confirmation; second press (when in confirm-delete state) executes the DELETE SQL against CARDDEMO.TRANSACTION_TYPE

Key	Action
F5	Save changes — when a record is not found, initiates new record creation flow; when edits have been validated and confirmed (TTUP-CHANGES-OK-NOT-CONFIRMED state), executes the UPDATE or INSERT SQL to persist the transaction type data
F6	Add new record (displayed on screen as a function key label but primary add flow is driven through F5 after record-not-found state; F6 label is shown but controlled as dark/inactive in most states)
F12	Cancel the current operation — cancels a pending update, delete confirmation, or new record creation and returns to the appropriate prior state (back to search prompt or re-displays original record values)